Lexical Analyzer

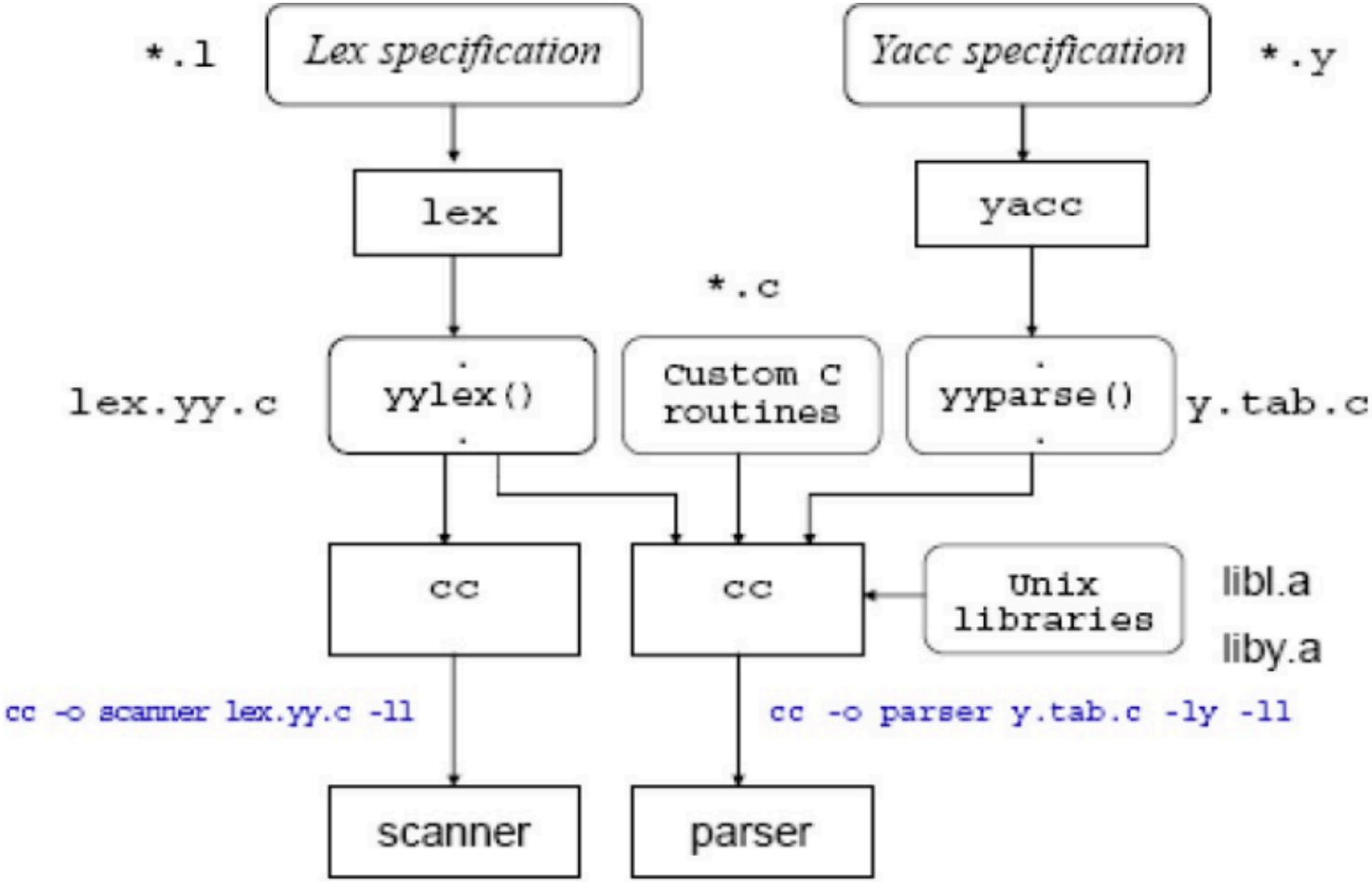
LEX (Lex/Yacc , flex/bison)

Ahmad Yoosofan

Compiler course

University of Kashan

- https://yoosofan.github.io/course/compiler.html



https://web.cs.hacettepe.edu.tr/~bbm301/lectures/lecture3_lex.pdf

Install and Compile Lex File

```
%%
[0-9]+                  {printf("INT:%s\n",yytext);}
[a-zA-Z_][a-zA-Z_0-9]* {printf("ID :%s\n",yytext);}
.    ;
\n   ;
%%

int main(){
  yylex();
  printf("\n");
  return 0;
}
```

```
# Install flex in Ubuntu

sudo apt install g++

sudo apt install flex

# Compile Lex File

flex 01.count_words.l

gcc lex.yy.c -lfl

./a.out < count_words_input_1.txt
```

```c
#line 3 "lex.yy.c"

#define  YY_INT_ALIGNED short int

/* A lexical scanner generated by flex */

#define FLEX_SCANNER
#define YY_FLEX_MAJOR_VERSION 2
#define YY_FLEX_MINOR_VERSION 6
#define YY_FLEX_SUBMINOR_VERSION 4
#if YY_FLEX_SUBMINOR_VERSION > 0
#define FLEX_BETA
#endif

/* First, we deal with  platform-specific or compiler-specific issues. */

/* begin standard C headers. */
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>

/* end standard C headers. */
```

Count words(I)

```
%%
[0-9]+                  {printf("INT:%s\n",yytext);}
[a-zA-Z_][a-zA-Z_0-9]* {printf("ID :%s\n",yytext);}
.    ;
\n   ;
%%

int main(){
  yylex();
  printf("\n");
  return 0;
}
```

```
aDDDaaHHbm412abAmns12bm2
aabbbbbb2341abcccccc
79aaaaabbbbbm8888888882
ab6669bm8888884
ab6669bm888888
abbaldkjA
1ab 238234234 2342.234234 34 3.3
abm 9bm88888
66abm
```

```
ID :aDDDaaHHbm412abAmns12bm2
ID :aabbbbbb2341abcccccc
INT:79
ID :aaaaabbbbbm8888888882
ID :ab6669bm8888884
ID :ab6669bm888888
ID :abbaldkjA
INT:1
ID :ab
INT:238234234
INT:2342
INT:234234
INT:34
INT:3
INT:3
ID :abm
INT:9
ID :bm88888
INT:66
ID :abm
```

Count words(II)

```
%%
[0-9]+                  {printf("int:%s\n",yytext);}
[a-zA-Z_][a-zA-Z_0-9]+ {printf("ID :%s\n",yytext);}
[1-4]+abc*              {printf("2  :%s\n",yytext);}
.    ;
\n   ;
%%
int main(){
  yylex();
  printf("\n");
  return 0;
}
```

```
aDDDaaHHbm412abAmns12bm2
aabbbbbb2341abcccccc
79aaaaabbbbbm8888888882
ab6669bm8888884
ab6669bm888888
abbaldkjA
1ab 238234234 2342.234234 34 3.3
abm 9bm88888
66abm
```

```
lex$ ./a.out < count_words_input_1.tx

ID :aDDDaaHHbm412abAmns12bm2
ID :aabbbbbb2341abcccccc
int:79
ID :aaaaabbbbbm8888888882
ID :ab6669bm8888884
ID :ab6669bm888888
ID :abbaldkjA
2  :1ab
int:238234234
int:2342
int:234234
int:34
int:3
int:3
ID :abm
int:9
ID :bm88888
int:66
ID :abm
```

Count words(III)-input(I)

```
%%
^[0-9]+a+b+          {printf("1 %s\n", yytext);}
[1-4]+abc*           {printf("2 %s\n", yytext);}
[a-zA-Z]*[4-5]+      {printf("3 %s\n", yytext);}
[^0-8]bm8*$          {printf("4 %s\n", yytext);}
.                       ;
\n                      ;
%%
int main(){ yylex();printf("\n");return 0;}
```

```
aDDDaaHHbm412abAmns12bm2
aabbbbbb2341abcccccc
79aaaaabbbbbm8888888882
ab6669bm8888884
ab6669bm888888
abbaldkjA
1ab 238234234 2342.234234 34 3.3
abm 9bm88888
66abm
```

```
lex$ flex 04.count_words.l
lex$ gcc lex.yy.c -lfl
lex$ ./a.out < count_words_input_1.txt

3 aDDDaaHHbm4
2 12ab
2 2341abcccccc
1 79aaaaabbbbb
3 4
4 9bm888888
1 1ab
3 4
3 4
3 4
3 4
3 4
3 4
4 9bm88888
1 66ab
```

Count words(III)-input(II)

```
%%
^[0-9]+a+b+          {printf("1 %s\n", yytext);}
[1-4]+abc*           {printf("2 %s\n", yytext);}
[a-zA-Z]*[4-5]+      {printf("3 %s\n", yytext);}
[^0-8]bm8*$          {printf("4 %s\n", yytext);}
.                       ;
\n                      ;
%%
int main(){ yylex();printf("\n");return 0;}
```

```
aDDDaaHHbm412abAmns12bm2
aabbbbbb2341abcccccc
79aaaaabbbbbm8888888882
ab6669bm8888884
ab6669bm888888
abbaldkjA
1ab 238234234 2342.234234 34 3.3
abm 9bm88888
66abm

```

```
lex$ flex 04.count_words.l
lex$ gcc lex.yy.c -lfl
lex$ ./a.out < count_words_input_2.txt

3 aDDDaaHHbm4
2 12ab
2 2341abcccccc
1 79aaaaabbbbb
3 4
4 9bm888888
1 1ab
3 4
3 4
3 4
3 4
3 4
3 4
4 9bm88888
1 66ab
```

Simple Number

```
%%
^aa             {printf("AAAAAAA\n"); }
^ss             {printf("SSSSSSS\n"); }
[0-9]+          {printf("Integer:: %s\n",yytext);}
[0-9]+\.[0-9]+   {printf("Real :: %s\n",yytext);}
.                       ;
\n                      ;
%%
int main(){
    yylex();
    printf("\n");
    return 0;
}
```

```
12 234 434.45 676.45
eer
df
aa   abc bb aa
ss aa aaa  ss abcabc  654  sdfsf 123 lsdkjfsl
aa   ss
```

```
lex$ flex 05.simpleNumber.l
lex$ gcc lex.yy.c -lfl
lex$ ./a.out < 05.simpleNumber_input.t

Integer:: 12
Integer:: 234
Real :: 434.45
Real :: 676.45
AAAAAAA
SSSSSSS
Integer:: 654
Integer:: 123
AAAAAAA
```

Number of Characters

```
%{
int numberOfInt=0   , numberOfDouble=0 ,
  numberOfString = 0 ,  numberOfLine =0 ;
int numberOfDot =0 , numberOfOther=0;
%}
%%
\n                {numberOfLine++;}
[0-9]+            {numberOfInt++;}
[0-9]*\.[0-9]+    {numberOfDouble++;}
\.                {numberOfDot++;}
[a-zA-Z]+         {numberOfString++;}
[ \t]             {}
.                 {numberOfOther++;}
%%
int main(){
  yylex();
  printf("Int::%d\n", numberOfInt);
  printf("Double::%d\n", numberOfDouble);
  printf("Double::%d\n", numberOfString);
  printf("Line::%d\n", numberOfLine);
  printf("Dot::%d\n", numberOfDot);
  printf("Other::%d\n", numberOfOther);
}
```

```
aaabm12Amns12bm8
aabbbbbb2345abcccccc
79bm8888888882
ab666bm888888
bm9abbbbbb88abbbbbb
bm
```

```
lex$ flex 08.numberOfCharacters.l
lex$ gcc lex.yy.c -lfl
lex$ ./a.out < 08.numberOfCharacters_input.txt

Int::10
Double::0
Double::12
Line::6
Dot::0
Other::0
```

Parts of Speech(I)

```
%%
[\t ]+   {/* ignore white space */ ;}
aaa|bb {printf("%s :::: mytest\n",yytext);}
is|am|are|were|was|be {printf("%s: is a verb\n", yytext);}
being|been|do|does  {printf("%s: is a verb\n", yytext);}
did|will|would|should {printf("%s: is a verb\n", yytext);}
can|could|has|have {printf("%s: is a verb\n", yytext);}
had|go  {printf("%s: is a verb\n", yytext);}
very|simply|gently {printf("%s: is an adverb\n", yytext);}
quietly|calmly|angrily {printf("%s: is an adverb\n",yytext);}
to|from|behind { printf("%s: is a preposition\n", yytext);}
above|below  {printf("%s: is a preposition\n", yytext);}
between|below {printf("%s: is a preposition\n", yytext);}
if|then|and {printf("%s: is a conjunction\n", yytext);}
but|or {printf("%s: is a conjunction\n", yytext);}
their|my|your {printf("%s: is an adjective\n", yytext);}
his|her|its  {printf("%s: is an adjective\n", yytext);}
I|you|he|she {printf("%s: in a pronoun\n", yytext);}
we|they {printf("%s: in a pronoun\n", yytext);}
[a-zA-Z]+ {printf("%s: don't recognize, a noun\n",yytext);}
\&.|\n    { ECHO; /* normal default anyway */ }
%%
int main(){ yylex(); }
```

```
aaa bbb is  am above
I you 2376429734 2342.23423
```

```
aaa :::: mytest
bbb: don't recognize, a nou
is: is a verb
am: is a verb
above: is a preposition

I: in a pronoun
you: in a pronoun
23764297342342.23423
```

Parts of Speech(II)

```
verb am
verb is
verb are
prep at
prep of
prep in
I am an instructor.
prep to
noun orange window
verb have has
noun university Kashan Ahmad
They have a situation.
This orange is not red.
pron I  you
My name is Ahmad.
I'm an instructor at
the university of Kashan.
You are students.
```

```
I:don't recognize
am:verb
an:don't recognize
instructor:don't recognize
They:don't recognize
have:verb
a:don't recognize
situation:don't recognize
This:don't recognize
orange:noun
is:verb
not:don't recognize
red:don't recognize
My:don't recognize
name:don't recognize
is:verb
Ahmad:noun
I:pronoun
m:don't recognize
an:don't recognize
instructor:don't recognize
at:preposition
the:don't recognize
university:noun
of:preposition
Kashan:noun
You:don't recognize
are:verb
students:don't recognize
```

Parts of Speech(III)

```
%{/* Word recognizer with a symbol table.*/
enum{
  LOOKUP = 0,
  /*default-looking rather than defining.*/
  VERB,ADJ,ADV,NOUN,PREP,PRON,CONJ, ART
};
int state=LOOKUP;
int add_word(int type, char *word);
int lookup_word(char *word);
%}
%%
\n {
  state = LOOKUP;
}/* end of line, return to default state */
^verb { state = VERB; }
^adj  { state = ADJ; }
^adv  { state = ADV; }
^noun { state = NOUN; }
^prep { state = PREP; }
^pron { state = PRON; }
^conj { state = CONJ; }
^article {state = ART; }
[a-zA-Z]+  {
  /*a normal word, define it or look it up*/
  if(state != LOOKUP)
  /* define the current word */
    add_word(state, yytext);
  else
```

```
    switch(lookup_word(yytext)){
      case VERB:
        printf("%s:verb\n", yytext); break;
      case ADJ:
        printf("%s:adjective\n", yytext);
        break;
      case ADV:
        printf("%s:adverb\n", yytext);
        break;
      case NOUN:
        printf("%s:noun\n", yytext); break;
      case PREP:
        printf("%s:preposition\n",yytext);
        break;
      case PRON:
        printf("%s:pronoun\n", yytext);
        break;
      case CONJ:
        printf("%s:conjunction\n", yytext);
        break;
      case ART:
        printf("%s:article\n", yytext);
        break;
      default:
        printf("%s",yytext);
        printf("don't recognize\n");
    }
}      /* [a-zA-Z]+  {  */
```

Parts of Speech(IV)

```
.  /* ignore anything else */ ;
%%
int main(){yylex();}
/*define a linked list of words and types*/
struct word{
  char *word_name;
  int word_type;
  struct word *next;
};

/*first element in word list*/
struct word *word_list;

extern void *malloc();

int add_word(int type, char *word){
  struct word *wp;
  if(lookup_word(word) != LOOKUP){
    printf("!!! warning: ");
    printf("%s already defined\n", word);
    return 0;
  }
  /* word not there, allocate a new entry
     and link it on the list
```

```
  */
  wp = (struct word *)
    malloc(sizeof(struct word));

  wp->next = word_list;
  /* have to copy the word itself as well*/

  wp->word_name = (char *)
    malloc(strlen(word)+1);

  strcpy(wp->word_name, word);
  wp->word_type = type;
  word_list = wp;return 1;/* it worked */
}

int  lookup_word(char *word){
  struct word *wp = word_list;/*
  search down the list looking for the word
  */
  for(; wp; wp = wp->next)
    if(strcmp(wp->word_name, word) == 0)
      return wp->word_type;
  return LOOKUP;    /* not found */
}
```

Some References

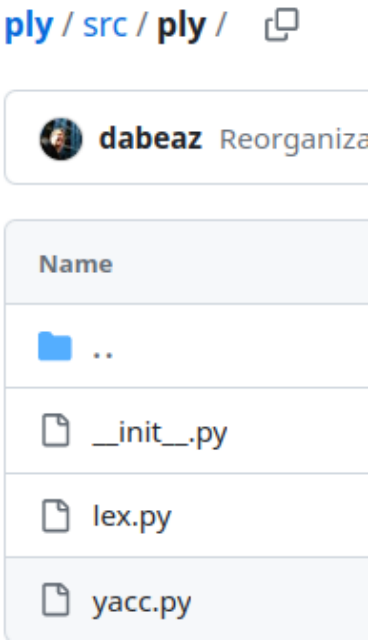
- Most Samples have been written by Ahmad Yoosofan (https://yoosofan.github.io)
- http://dinosaur.compilertools.net/#flex
- https://www.oreilly.com/library/view/lex-yacc/9781565920002/bi01.html
- http://compilerbook.org (https://www3.nd.edu/~dthain/compilerbook/)
- https://web.cs.hacettepe.edu.tr/~bbm301/lectures/lecture3_lex.pdf
- https://www.cs.utexas.edu/users/novak/lexpaper.htm
- https://silcnitc.github.io/lex.html
- https://www.opensourceforu.com/2017/04/lex-waving-the-magic-wand-of-regular-expressions/
- http://www.cs.columbia.edu/~aho/cs4115/Lectures/15-02-02.html

Similar Lexer and Parser

1. PLY
2. Lemon for SQLite
    1. https://www.sqlite.org/lang.html
    2. https://www.sqlite.org/pragma.html

PLY / lex part

- https://github.com/dabeaz/ply/
- www.dabeaz.com/ply/
- https://github.com/dabeaz/ply/tree/master/src/ply



Sample I

```
tokens = ('NUMBER','PLUS', 'MUL', 'ID')
t_PLUS    = r'\+'
t_MUL    = r'\*'
t_ID = r'[a-zA-Z_][a-zA-Z_0-9]*'
t_NUMBER = r'[0-9]+'

t_ignore = " \t"

import ply.lex as lex
lex.lex()

lex.input("5434+2882 + count34+   45")
while True:
  tok = lex.token()
  if not tok: break
  print(tok)
```

Sample II

```
tokens = ('NUMBER','PLUS')
t_PLUS    = r'\+'

def t_NUMBER(t):
  r'[0-9]+'
  t.value = int(t.value)
  return t

t_ignore = " \t"

def t_error(t):
  print("Illegal character '%s'" % t.value[0])
  t.lexer.skip(1)

import ply.lex as lex
lex.lex()
lex.input("*4+2")
while True:
  tok = lex.token()
  if not tok:  break
  print(tok)
```

Sample III

```
tokens = ('NUMBER','PLUS')
t_PLUS    = r'\+'

def t_NUMBER(t):
  r'[0-9]+'
  t.value = int(t.value)
  return t

t_ignore = " \t"

def t_error(t):
    print("Illegal character '%s'" % t.value[0])
    t.lexer.skip(1)

import ply.lex as lex
lex.lex()
lex.input("*4+2")
while True:
  tok = lex.token()
  if not tok:  break
  print(tok)
"""
lex$ python3 126.error.plus.py
Illegal character '*'
LexToken(NUMBER,4,1,1)
LexToken(PLUS,'+',1,2)
LexToken(NUMBER,2,1,3)
lex$
"""
```

Sample IV

```python
# 128.pluse.mul.py
tokens = ('NUMBER','PLUS', 'MUL')
t_PLUS    = r'\+'
t_MUL    = r'\*'

def t_NUMBER(t):
  r'[0-9]+'
  t.value = int(t.value)
  return t

t_ignore = " \t"

def t_error(t):
    print("Illegal character '%s'" % t.value[0])
    t.lexer.skip(1)

import ply.lex as lex
lex.lex()
lex.input("*445+23*34+123")

while True:
  tok = lex.token()
  if not tok:  break
  print(tok)

"""
lex$ python3 128.plus.mul.py
LexToken(MUL,'*',1,0)
LexToken(NUMBER,445,1,1)
LexToken(PLUS,'+',1,4)
LexToken(NUMBER,23,1,5)
LexToken(MUL,'*',1,7)
LexToken(NUMBER,34,1,8)
LexToken(PLUS,'+',1,10)
LexToken(NUMBER,123,1,11)
lex$
"""
```

Sample V

```python
tokens = ('NUMBER','PLUS', 'MUL', 'ID')
t_PLUS    = r'\+'
t_MUL    = r'\*'
t_ID = r'[a-zA-Z_][a-zA-Z_0-9]*'

def t_NUMBER(t):
  r'[0-9]+'
  t.value = int(t.value)
  return t

t_ignore = " \t"

def t_error(t):
    print("Illegal character '%s'" % t.value[0])
    t.lexer.skip(1)

import ply.lex as lex
lex.lex()
lex.input("wwe2+4+23*34+123")
```

```python
while True:
  tok = lex.token()
  if not tok:  break
  print(tok)
"""
lex$ python3 130.id.py
LexToken(ID,'wwe2',1,0)
LexToken(PLUS,'+',1,4)
LexToken(NUMBER,4,1,5)
LexToken(PLUS,'+',1,6)
LexToken(NUMBER,23,1,7)
LexToken(MUL,'*',1,9)
LexToken(NUMBER,34,1,10)
LexToken(PLUS,'+',1,12)
LexToken(NUMBER,123,1,13)
lex$
"""
```

Sample VI

```python
tokens = ('NUMBER','PLUS', 'MUL', 'ID', 'ASG')
t_PLUS    = r'\+'
t_MUL    = r'\*'
t_ID = r'[a-zA-Z_][a-zA-Z_0-9]*'
t_ASG = '='
t_NUMBER = r'[0-9]+'
t_ignore = " \t\n"

def t_error(t):
    print("Illegal character '%s'" % t.value[0])
    t.lexer.skip(1)

import ply.lex as lex
lex.lex()
lex.input("""wwe2 = 4+23 *34 +123
count+34+435
x=y
""")
while True:
  tok = lex.token()
  if not tok:  break
  print(tok)
```

```
"""
lex$ python3 132.assign.py
LexToken(ID,'wwe2',1,0)
LexToken(ASG,'=',1,5)
LexToken(NUMBER,4,1,7)
LexToken(PLUS,'+',1,8)
LexToken(NUMBER,23,1,9)
LexToken(MUL,'*',1,12)
LexToken(NUMBER,34,1,13)
LexToken(PLUS,'+',1,16)
LexToken(NUMBER,123,1,17)
LexToken(ID,'count',1,21)
LexToken(PLUS,'+',1,26)
LexToken(NUMBER,34,1,27)
LexToken(PLUS,'+',1,29)
LexToken(NUMBER,435,1,30)
LexToken(ID,'x',1,34)
LexToken(ASG,'=',1,35)
LexToken(ID,'y',1,36)
lex$
"""
```

Sample VII

```python
tokens = ('NUMBER','PLUS', 'MUL', 'ID', 'ASG',
  'COMP')
t_PLUS    = r'\+'
t_MUL    = r'\*'
t_ID = r'[a-zA-Z_][a-zA-Z_0-9]*'
t_ASG = '='
t_COMP = '=='
t_NUMBER = r'[0-9]+'
t_ignore = " \t\n"

def t_error(t):
    print("Illegal rrr '%s'" % t.value[0])
    t.lexer.skip(1)

import ply.lex as lex
lex.lex()
lex.input("""wwe2 ==  4+23 = *34 +123
count+34+435
x=y
""")
```

```python
while True:
  tok = lex.token()
  if not tok:  break
  print(tok)

"""
lex$ python3 132.assign.py
LexToken(ID,'wwe2',1,0)
LexToken(ASG,'=',1,5)
LexToken(NUMBER,4,1,7)
LexToken(PLUS,'+',1,8)
LexToken(NUMBER,23,1,9)
LexToken(MUL,'*',1,12)
LexToken(NUMBER,34,1,13)
LexToken(PLUS,'+',1,16)
LexToken(NUMBER,123,1,17)
LexToken(ID,'count',1,21)
LexToken(PLUS,'+',1,26)
LexToken(NUMBER,34,1,27)
LexToken(PLUS,'+',1,29)
LexToken(NUMBER,435,1,30)
LexToken(ID,'x',1,34)
LexToken(ASG,'=',1,35)
LexToken(ID,'y',1,36)
lex$
"""
```

Sample VIII

```python
tokens = ('NUMBER','PLUS', 'MUL', 'ID', 'ASG')
t_PLUS    = r'\+'
t_MUL    = r'\*'
t_ID = r'[a-zA-Z_][a-zA-Z_0-9]*'
t_ASG = '='

def t_NUMBER(t):
  r'[0-9]+'
  t.value = int(t.value)
  return t

t_ignore = " \t"

def t_newline(t):
  r'\n+'
  t.lexer.lineno += len(t.value)
  print('new line')

def t_error(t):
    print("Illegal character '%s'" % t.value[0])
    t.lexer.skip(1)

import ply.lex as lex
lex.lex()
lex.input("""wwe2 = 4+23 *34 +123
count+34+435
x=y
""")
while True:
  tok = lex.token()
  if not tok:  break
  print(tok)
"""
lex$ python3 134.newline.py
LexToken(ID,'wwe2',1,0)
LexToken(ASG,'=',1,5)
LexToken(NUMBER,4,1,7)
LexToken(PLUS,'+',1,8)
LexToken(NUMBER,23,1,9)
LexToken(MUL,'*',1,12)
LexToken(NUMBER,34,1,13)
LexToken(PLUS,'+',1,16)
LexToken(NUMBER,123,1,17)
new line
LexToken(ID,'count',2,21)
LexToken(PLUS,'+',2,26)
LexToken(NUMBER,34,2,27)
LexToken(PLUS,'+',2,29)
LexToken(NUMBER,435,2,30)
new line
LexToken(ID,'x',3,34)
LexToken(ASG,'=',3,35)
LexToken(ID,'y',3,36)
new line
"""
```

A calculator

```python
tokens = ('NUMBER','PLUS')
t_PLUS    = r'\+'

def t_NUMBER(t):
  r'[0-9]+'
  t.value = int(t.value)
  return t

t_ignore = " \t"

def t_error(t):
  print("Illegal character '%s'" % t.value[0])
  t.lexer.skip(1)

import ply.lex as lex
lex.lex()

"""
S → E
E → E + T
E → a
"""
```

```python
def p_s(p):
  'S : E'
  print('S → E ', p[1])

def p_e_e_a(p):
  'E : E PLUS NUMBER' # E → E + a
  p[0] = p[1] + p[3]
  print('E → E_1 + a :', 'E: ', end='')
  print(p[0], ' \t E1: ', p[1], end='')
  print('\t a: ', p[3])

def p_e_a(p):
  'E : NUMBER' # E → a
  p[0] = p[1]
  print('E → a :', p[1])

def p_error(p):
  print("Syntax error at '%s'" % p.value)

import ply.yacc as yacc;
yacc.yacc()

while True:
  s = input('calc > ')
  if s.strip()==':break
  yacc.parse(s)
```

END